

# Leaf-alias: real-time importance sampling of dynamic emissive textures

Jose Aguerre  
Universidad de la República  
Montevideo, Uruguay  
jpaguerre@fing.edu.uy

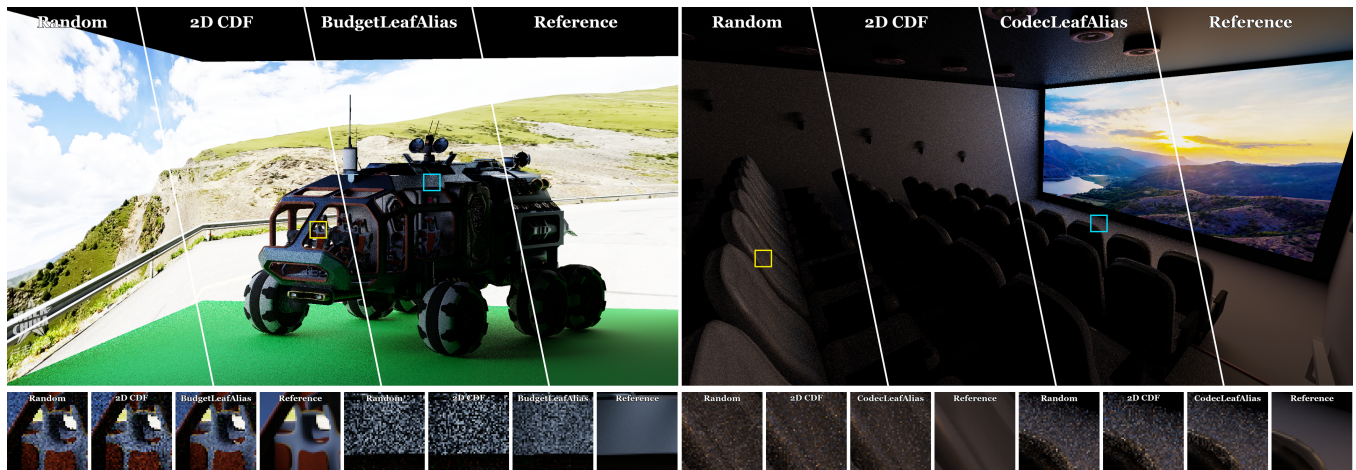


Figure 1: Two scenes lit by a single dynamic, video-driven emissive texture, rendered with our leaf-alias importance sampler. (Left) The Rover scene — a virtual-production LED wall displaying a looping road/mountain video panorama that lights the vehicle directly, sampled here with BudgetLeafAlias. (Right) The Cinema scene — a theater lit only by the animated sunset footage playing on its own screen, sampled with CodecLeafAlias. Each panel compares, left to right: Random sampling, 2D CDF inversion, our technique, and a converged reference, all at their calibrated real-time budgets ( $\approx 33$  ms).

## Abstract

We present a method for fast importance sampling HDR textured lights that allows real-time video-driven lighting. Our method partitions a light’s texture into a budgeted, variance driven quadtree and builds an alias table over the resulting leaves, avoiding the cost of a dense per-frame CDF while still capturing the texture’s overall energy distribution. We present two variants: one that reuses the hierarchical structures already present in the metadata of modern codec video streams, and another that builds a budgeted GPU-friendly quadtree to fit the entire sampling structure in L1 cache. Compared to the dense 2D CDF-inversion baseline, at a matched real-time frame-time budget our implemented technique reduces RMSE by 9–53% depending on scene, letting the saved sampling cost be reinvested in additional light samples. It also reduces memory

consumption and transfers by several orders of magnitude when compared to state of the art samplers.

## CCS Concepts

• Computing methodologies → Reflectance modeling.

## Keywords

These are the keywords

## ACM Reference Format:

Jose Aguerre. 2025. Leaf-alias: real-time importance sampling of dynamic emissive textures. *ACM Trans. Graph.* 1, 1 (September 2025), 5 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

## 1 Introduction

Importance sampling of emissive textures is essential for efficient image-based lighting, and increasingly critical in production workflows where video is used directly as a dynamic light source in virtual production and VFX. A single emissive texture lets a renderer reproduce the directional detail of a real environment, far beyond what analytic lights can approximate. Traditionally this texture has been static, captured once and reused for an entire sequence. Increasingly, though, it is video: LED-wall virtual production, AR/VR object insertion, and video-based environment lighting

Author’s Contact Information: Jose Aguerre, Universidad de la República, Montevideo, Uruguay, [jpaguerre@fing.edu.uy](mailto:jpaguerre@fing.edu.uy).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.  
ACM 1557-7368/2025/9-ART  
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

all require illumination that changes every frame, not a single fixed capture.

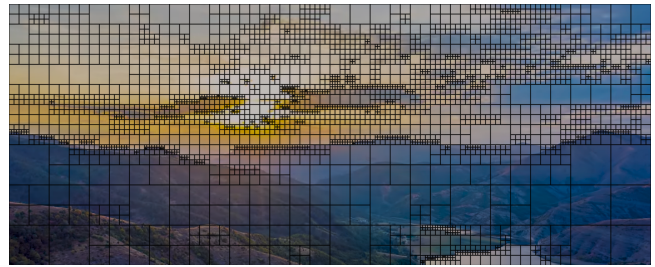
Traditional texture light sampling techniques are designed for static images. They rely on the offline building of large sampling structures that are reused for rendering each frame as long as the image doesn't change. The most common example of such strategy is how rendering and ray tracing toolboxes deal with HDR environment maps. The state-of-the-art approach is to build a two-dimensional piecewise constant distribution from the texture's luminance: a marginal CDF over rows and a per-row conditional CDF. The sampler is then inverted via binary search at sample time, which is expensive for large textures. A more recent and well-adopted approach is to generate a mipmap-style hierarchical importance map [1]. At sample time, a hierarchical descent through a mip pyramid of the texture is performed, recursively choosing one of four children at each level in proportion to their summed weight. This technique is also expensive and requires several iterations per sample, although it is more GPU friendly because the number of iterations is always the same for each sample, unlike in the binary search. Moreover, the mipmap can be fastly constructed using the existing dedicated hardware on modern GPUs. Both techniques require at least  $O(N)$  memory for the sampler, with  $N$  being the number of pixels.

Another less used but known technique for sampling emissive textures is the Alias method [2]. This technique draws from an arbitrary discrete distribution in  $O(1)$  time after an  $O(N)$  one-time build, at the cost of losing the CDF's implicit spatial coherence: two adjacent table buckets can alias to two arbitrary, unrelated locations in the original distribution. This fact is known to disrupt the stratification or discrepancy-reduction of the input point set. It also requires  $O(N)$  memory to store the correlation between every pixel. The process for building the Alias table, despite being  $O(N)$ , is difficult to optimize for real-time construction [3].

We present a new sampling method that is fast to build (under 1 millisecond for 4k HDR images), produces a sampler that is tiny, and is  $O(1)$  to sample without disrupting the stratification of the Quasi Monte Carlo input. The core idea is to reuse the variance-driven image partition that is already present in modern video codecs, which already compute hierarchical spatial partitions that adapt to local signal variance [Uhrina et al. 2024]. These structures implicitly encode the spatial distribution of luminance and high-frequency content (see Fig. 2). We also propose a variant that builds the partition tree on a budgeted number of leaves, using a Greedy/best-first tree-structured resource allocation [Minasny et al. 2007; Zhou et al. 2024]. Both techniques show consistent quality gains when rendering video light textures in real-time (33 milliseconds per frame), since they allow executing more texture samples without introducing stratification error.

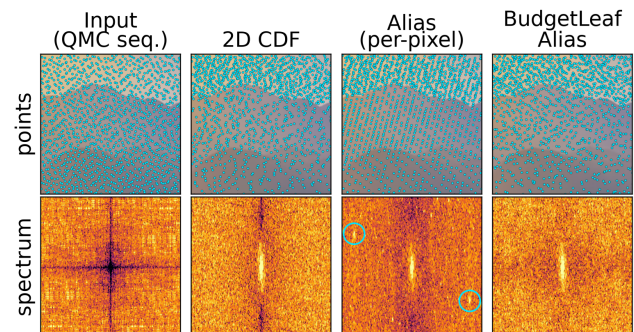
## 2 Previous works

Importance sampling of emissive textures is a well-studied problem in the graphics literature in the context of image-based lighting. Most of the works focus on pre-processing the images to allow for exact or near-exact sampling without introducing bias [Lawrence et al. 2005]. Several methods choose to simplify the textured light into a set of point or area lights [Agarwal et al. 2003], allowing



**Figure 2: An example of a variance-driven quadtree built by the VP9 video codec. High variance zones are refined into a dense mesh of small cells.**

for real-time sampling and faster convergence but at the cost of more error in the solution. Cline et al. [Cline et al. 2009] compare several tabular inversion methods, noting the computational order of building each sampler structure and sampling each point. They analyze the 2D CDF inversion via binary search and the per-pixel Alias method, where they flag the disruption of the stratification of the input point set. Per-pixel Alias's raster-flattened table produces a real, measurable periodic banding pattern (Figure 3) that becomes clear in the sample set's power spectrum. We show that our LeafAlias method overcomes those issues because the partition structure helps with stratification.



**Figure 3: 2D power spectrum of samples from 2D CDF inversion, plain per-pixel alias, and our LeafAlias, from the same Halton input. Per-pixel alias shows a sharp, isolated off-axis peak (flatness 0.270, peak/median 16,113) – a periodic banding artifact from its flat array index resonating with the input sequence's base-2 digit structure. Both CDF inversion and ours stay close to the input's own clean spectrum.**

Despite being  $O(1)$  to sample, the per-pixel Alias method is heavy in memory consumption and in the construction of the tables via Vose's technique. Nevertheless, the Alias method is widely used in mesh lights sampling: geometry with thousands of emissive triangles. Moreau et al. [Moreau et al. 2019] maintain a light-sampling BVH incrementally across frames for thousands of moving lights. Bitterli et al.'s ReSTIR [Bitterli et al. 2020] resamples across spatial and temporal neighbors to pick among many dynamic lights at a fraction of the cost of resampling from scratch. Conty Estevez and Kulla [Conty Estevez and Kulla 2018] adaptively split a light tree

per shading point rather than committing to one fixed cut. At that scale of thousands of triangles, a plain alias table over per-triangle flux is common practice and cheap to rebuild every frame.

Regarding the optimization of the sampling structures, Ergun et al. [Ergun et al. 2012] split an environment map into a kd-tree via the greedy partition strategy, to a predefined block count, then fit the sorted leaf probabilities with a closed-form curve for direct inversion. They report gains in memory compression, higher speed and accuracy than CDF inversion at matched performance for static environment maps. One disadvantage of using kd-trees is that evaluating the pdf of the map direction when sampled from BRDF in multiple importance sampling (MIS) requires traversing the tree. Lambers [Lambers 2022] proposes a fixed-grid sampler and notes that replacing the grid with a variance-driven quadtree might improve their results. Rodrigues et al. [Rodriguez-Pardo et al. 2023] train a neural normalizing flow that allows both sampling and evaluating the pdf, showing considerable performance gains at the cost of pre-training the structure for the static map. Wu [Wu 2025] also preprocess the map by dividing the emissive and non-emissive parts into separate structures.

The previously described techniques focus mainly in preprocessing static images to improve sampling efficiency, but not dynamic maps. Some previous works address video-driven lighting: Havran et al. [Havran et al. 2005] and Iorns and Rhee [Iorns and Rhee 2015] both light scenes from captured video environment maps, but neither targets real-time per-frame resampling of the video itself as the importance distribution. Lu et al. [Lu et al. 2013] rebuild a tabulated per-cube-face CDF in real-time, noting it as an expensive part of the process. Kronander et al. [Kronander et al. 2014] rely on temporal sample reuse to improve convergence of video-based lighting.

Attempts to make the per-frame rebuild itself cheaper on a GPU point the same direction: Morrical and Zellmann [Morrical and Zellmann 2021] accelerate 2D CDF inversion with ray-tracing hardware at high construction cost and non-trivial implementation complexity. Lehmann et al. [Lehmann et al. 2021] study GPU-specific weighted sampling and alias-table construction directly.

Current open-source renderers mostly rely on dense CDF or mipmapped samplers. Falcor [NVIDIA 2024] uses a hierarchical mip-pyramid importance map over the environment texture’s luminance and, separately, the triangle-scale alias table for mesh lights. Mitsuba 3 [Nimier-David et al. 2019] builds a Hierarchical2D warp over the environment map – the same mip-pyramid family as Falcor’s. pbrt-v4 [Pharr et al. 2023] uses a dense CDF distribution, inverted via binary search. Blender Cycles [Blender Foundation 2024] builds the same kind of dense marginal/conditional CDF over a downsampled background map. Every one of the four either assumes the environment texture is built once and reused.

### 3 Method

The LeafAlias technique applies to any light source described by a 2D radiance texture  $L(u, v)$  over a parameter domain  $(u, v) \in [0, 1]^2$  – an environment map, or the emission texture of an area light. As with any textured light, variance is minimized when the sampling density matches radiance,  $p(y) \propto L(y)$ . We approximate this with a piecewise-constant density over a partition of the domain into

disjoint leaf rectangles  $\{R_i\}_{i=1}^N$  – for now assumed given; Sections 3.2 and 3.3 describe the two ways we actually construct it. Each leaf carries a single weight proportional to its flux: its area in the parameter domain times its average radiance:

$$w_i = A_i \bar{L}_i, \quad \bar{L}_i = \frac{1}{A_i} \int_{R_i} L(u, v) dA. \quad (1)$$

Normalizing over all leaves gives a piecewise-constant density over the parameter domain and a per-leaf selection probability:

$$p(y) = \frac{\bar{L}_i}{W} \text{ for } y \in R_i, \quad p_i = \frac{w_i}{W}, \quad W = \sum_{j=1}^N w_j. \quad (2)$$

Converting  $p(y)$  to whatever measure a particular light type actually samples with respect to (solid angle for a distant environment map or projected solid angle for a local area light) is a standard change-of-variables applied at evaluation time, and is independent of the sampling structure described below.

Given the partition, we flatten it into an array of  $N$  leaves, each storing its rectangle and weight  $w_i$  from Equation 1. We build a Walker/Vose alias table [Vose 1991] over this array once, in  $O(N)$ : leaves below the mean weight are paired with leaves above it, the excess of the latter absorbs the deficit of the former, and the process repeats until every leaf holds a threshold and, where needed, the index of the leaf it aliases to (Algorithm 1). Unlike a dense CDF, the resulting table needs no search at sample time – selecting a leaf is an  $O(1)$  array lookup.

---

#### Algorithm 1 Build the alias table over $N$ leaves

---

```

1:  $W \leftarrow \text{mean}(w_1, \dots, w_N)$ 
2: partition leaf indices into small ( $w_i < W$ ) and large ( $w_i \geq W$ )
3: while small and large both non-empty do
4:    $s \leftarrow \text{pop}(\text{small}); \quad l \leftarrow \text{pop}(\text{large})$ 
5:    $\text{threshold}[s] \leftarrow w_s/W; \quad \text{alias}[s] \leftarrow l$ 
6:    $w_l \leftarrow w_l - (W - w_s)$  {transfer  $l$ 's excess weight to  $s$ }
7:   push  $l$  back onto small or large by its updated  $w_l$ 
8: end while
9: for remaining  $i$  in small  $\cup$  large do
10:    $\text{threshold}[i] \leftarrow 1$ 
11: end for
```

---

### 3.1 Sampling

To draw a light sample we use three uniform numbers  $\xi_1, \xi_2, \xi_3$  (Algorithm 2).  $\xi_1$  selects a bucket in  $O(1)$ ; comparing its leftover fractional part against the bucket’s threshold decides whether we keep that leaf or its alias partner.  $\xi_2, \xi_3$  place the sample uniformly inside the chosen leaf’s rectangle. The sample’s pdf is read directly from its leaf’s stored average luminance via Equation 2 – not re-derived from the alias table. Radiance is read from the texture at the sampled  $(u, v)$ .

Combining light sampling with BRDF sampling via MIS requires the reverse query: given a direction chosen by BRDF sampling that happens to hit the light at some  $(u, v)$ , the probability  $p(y)$  is needed. Rather than searching the partition, we maintain a small auxiliary lookup matrix that maps a coarsened  $(u, v)$  coordinate directly to its leaf index, at a fixed fraction of the texture’s resolution

**Algorithm 2** Draw one light sample

---

```

1:  $i \leftarrow \lfloor \xi_1 N \rfloor$ ;  $f \leftarrow \xi_1 N - i$ 
2: if  $\xi_2 < \text{threshold}[i]$  then
3:    $\text{leaf} \leftarrow i$ 
4: else
5:    $\text{leaf} \leftarrow \text{alias}[i]$ 
6: end if
7:  $(u, v) \leftarrow \text{lerp}(R_{\text{leaf.min}}, R_{\text{leaf.max}}, (f, \xi_3))$ 
8: return  $(u, v)$ ,  $p(y) = \bar{L}_{\text{leaf}}/W$  (Eq. 2),  $L(u, v)$ 

```

---

matching the partition's smallest possible leaf (e.g. one entry per  $8 \times 8$  block). Every coordinate covered by a given leaf is written to the same entry at construction time, so the lookup is an  $O(1)$  array read that reproduces the exact same  $\bar{L}_i/W$  used at sampling time regardless of that leaf's actual size, keeping the resulting MIS weight unbiased. We also tried replacing this exact lookup with a single scene-wide average density: across our test scenes this produced no perceptible difference in the rendered images, but it does introduce bias, since the substituted value no longer matches the technique's actual density at that point.

### 3.2 CodecLeafAlias

Building the partition can be costly, especially for dynamic content that changes every few milliseconds. CodecLeafAlias avoids it entirely when the texture arrives as a compressed video frame. Modern codecs (HEVC, AV1, VP9) already split each frame into a coding-tree hierarchy driven by local signal variance – structurally the same quadtree our leaves need, produced as a side effect of decoding rather than an extra pass. We take each coding-tree leaf's block directly as one of our leaf rectangles  $R_i$ , and estimate  $\bar{L}_i$  from the block's own zero-frequency (DC) transform coefficient rather than summing pixels: the DCT/DST's DC term is, up to a fixed normalization constant, exactly the block's mean sample value, and is already computed and present in decoder metadata. The resulting leaf array feeds directly into the alias-table construction.

In our experiments, we encode the dynamic emissive textures as HDR video using libvpx's VP9 Profile 2 and extract the superblock for each frame via decoder statistics. We configure the encoder to always use variance-based superblock partitioning. Encoder settings that instead search or reuse partitions (full rate-distortion search, predicted neural partitions, etc) could still supply a usable partition by combining each block's motion vector with its keyframe's already-computed superblocks. This strategy results in trees of varying number of leaves, ranging from 4000 to up to 20000 depending on the content. We call this method CodecLeafAlias. Since the sampling algorithms are memory-bound operations, we go further by defining a new partition mechanism that allows to chose a budgeted number of leaves to reduce memory needs and improve coherence. We call this other method BudgetLeafAlias.

### 3.3 BudgetLeafAlias

BudgetLeafAlias constructs the quadtree directly from the texture under a hard leaf-count budget  $B$  (in our implementation, matched to what fits alongside the alias table in a modern GPU's L1 data cache; Section 3.4). Rather than recursing to a fixed depth

or a fixed per-node threshold, we spend the budget greedily (Algorithm 3): a priority queue always refines whichever pending block currently contributes the most summed-squared error to the piecewise-constant approximation,  $\text{Var}(R) \cdot A(R)$ , until the budget is exhausted or no block has positive variance left to resolve. This is the standard greedy construction for a budget-constrained piecewise-constant approximation: unlike a fixed-depth or fixed-threshold rule, it spends every one of its  $B$  leaves wherever the source image actually has the most error left to remove.

**Algorithm 3** Build a budgeted quadtree of  $\leq B$  leaves

---

```

1:  $\text{queue} \leftarrow \{\text{root}\}$ ;  $\text{leaves} \leftarrow \emptyset$ 
2: while  $\text{queue} \neq \emptyset$  and  $|\text{leaves}| + |\text{queue}| + 3 \leq B$  do
3:    $R \leftarrow \text{pop block with max } \text{Var}(R) \cdot A(R) \text{ from } \text{queue}$ 
4:   for each of the 4 children } C \text{ of } R \text{ do}
5:     if  $C$  is  $8 \times 8$  texels or smaller, or  $\text{Var}(C) = 0$  then
6:        $\text{leaves} \leftarrow \text{leaves} \cup \{C\}$ 
7:     else
8:        $\text{queue} \leftarrow \text{queue} \cup \{C\}$ 
9:     end if
10:  end for
11: end while
12:  $\text{leaves} \leftarrow \text{leaves} \cup \text{queue}$ 

```

---

### 3.4 Implementation

We implement both variants in NVIDIA's Falcor rendering engine and test using a full path tracer with Quasi Monte Carlo sampling and MIS. We construct the LeafAlias tables and partitions in CPU, and pass the results to GPU in each frame. All the precomputation is done in under 1 millisecond per frame. We then implement the sampling method in the Slang Shading Language. In order to reduce GPU memory usage and improve coherency, we spent some effort in making the sampling structures as small as possible. Each leaf is packed into a 16-byte record – its rectangle as two packed 16-bit corner coordinates, the alias threshold and redirect index as a half-float and a 16-bit index, and  $\bar{L}_i$  as a half-float. For BudgetLeafAlias, we size  $B$  so the combined leaf and node buffers fit within roughly half of the GPU's per-core L1 cache, keeping the whole sampling structure resident there during traversal instead of round-tripping to global memory on every sample. This compactness helps improving performance when compared to other memory bound samplers like dense CDF or per-pixel alias table.

## 4 Results

These are the results

## 5 Conclusions and future work

These are the conclusions

## References

- Sameer Agarwal, Ravi Ramamoorthi, Serge Belongie, and Henrik Wann Jensen. 2003. Structured Importance Sampling of Environment Maps. In *ACM SIGGRAPH 2003 Papers*. 605–612.
- Benedikt Bitterli, Chris Wyman, Matt Pharr, Peter Shirley, Aaron Lefohn, and Wojciech Jarosz. 2020. Spatiotemporal Reservoir Resampling for Real-Time Ray Tracing with

- Dynamic Direct Lighting. *ACM Transactions on Graphics* 39, 4, Article 148 (2020), 148:1–148:17 pages. doi:10.1145/3386569.3392481
- Blender Foundation. 2024. Cycles: Open Source Production Rendering. <https://github.com/blender/cycles>.
- David Cline, Anshuman Razdan, and Peter Wonka. 2009. A Comparison of Tabular PDF Inversion Methods. *Computer Graphics Forum* 28, 1 (2009), 154–160. doi:10.1111/j.1467-8659.2008.01197.x
- Alejandro Conty Estevez and Christopher Kulla. 2018. Importance Sampling of Many Lights with Adaptive Tree Splitting. *Proceedings of the ACM on Computer Graphics and Interactive Techniques* 1, 2 (2018), 25:1–25:17.
- Serkan Ergun, Murat Kurt, and Aydın Öztürk. 2012. Real-time Kd-tree Based Importance Sampling of Environment Maps. In *Proceedings of the Spring Conference on Computer Graphics (SCCG)*, Roman Duriković and Carol O'Sullivan (Eds.). Comenius University Press, Budmerice, Slovakia, 84–91.
- Vlastimil Havran, Miloslaw Smyk, Grzegorz Krawczyk, Karol Myszkowski, and Hans-Peter Seidel. 2005. Interactive System for Dynamic Scene Lighting using Captured Video Environment Maps. In *Rendering Techniques*. 31–42.
- Thomas Iorns and Taehyun Rhee. 2015. Real-time Image Based Lighting for 360-degree Panoramic Video. In *Image and Video Technology*. Springer, 139–151.
- Joel Kronander, Johan Dahlin, Daniel Jönsson, Manon Kok, Thomas B Schön, and Jonas Unger. 2014. Real-time video based lighting using GPU raytracing. In *2014 22nd European Signal Processing Conference (EUSIPCO)*. IEEE, 1627–1631.
- Martin Lambers. 2022. Parameterization-Independent Importance Sampling of Environment Maps. *arXiv preprint arXiv:2208.10815* (2022).
- Jason Lawrence, Szymon Rusinkiewicz, and Ravi Ramamoorthi. 2005. Adaptive Numerical Cumulative Distribution Functions for Efficient Importance Sampling. In *Rendering Techniques*. 11–20.
- Hans-Peter Lehmann, Lorenz Hübshle-Schneider, and Peter Sanders. 2021. Weighted Random Sampling on GPUs. *arXiv:2106.12270*
- Heqi Lu, Romain Pacanowski, and Xavier Granier. 2013. Real-Time Importance Sampling of Dynamic Environment Maps. In *Eurographics 2013 - Short Papers*. The Eurographics Association, Girona, Spain, 65–68.
- Budiman Minasny, Alex B. McBratney, and Dennis J.J. Walvoort. 2007. The variance quadtree algorithm: Use for spatial sampling design. *Computers & Geosciences* 33, 3 (2007), 383–392. doi:10.1016/j.cageo.2006.08.009
- Pierre Moreau, Matt Pharr, and Petrik Clarberg. 2019. Dynamic Many-Light Sampling for Real-Time Ray Tracing. In *High-Performance Graphics 2019 - Short Papers*. The Eurographics Association. doi:10.2312/hpg.20191191
- Nate Morrical and Stefan Zellmann. 2021. Inverse Transform Sampling Using Ray Tracing Hardware. In *Ray Tracing Gems II: Next Generation Real-Time Rendering with DXR, Vulkan, and OptiX*. Springer, 625–641.
- Merlin Nimier-David, Delio Vicini, Tizian Zeltner, and Wenzel Jakob. 2019. Mitsuba 2: A Retargetable Forward and Inverse Renderer. *ACM Transactions on Graphics (Proceedings of SIGGRAPH Asia)* 38, 6 (2019). doi:10.1145/3355089.3356498
- NVIDIA. 2024. Falcor: Real-Time Rendering Framework. <https://github.com/NVIDIAGameWorks/Falcor>.
- Matt Pharr, Wenzel Jakob, and Greg Humphreys. 2023. *Physically Based Rendering: From Theory to Implementation* (4th ed.). MIT Press.
- Carlos Rodriguez-Pardo, Javier Fabre, Elena Garces, and Jorge Lopez-Moreno. 2023. NEnv: Neural environment maps for global illumination. In *Computer graphics forum*, Vol. 42. Wiley Online Library, e14883.
- Miroslav Uhrina, Lukas Sevcik, Juraj Bienik, and Lenka Smatanova. 2024. Performance comparison of VVC, AV1, HEVC, and AVC for high resolutions. *Electronics* 13, 5 (2024), 953.
- Michael D. Vose. 1991. A Linear Algorithm for Generating Random Numbers with a Given Distribution. *IEEE Transactions on Software Engineering* 17, 9 (1991), 972–975. doi:10.1109/32.92917
- Yu-Ting Wu. 2025. Efficient Environment Map Rendering Based on Decomposition. In *Computer Graphics Forum*, Vol. 44. Wiley Online Library, e15264.
- Yanzhi Zhou, Pengfei Lin, Hailong Liu, Weipeng Zheng, Xiaoxia Li, and Wenzhou Zhang. 2024. Fast and flexible spatial sampling methods based on the Quadtree algorithm for ocean monitoring. *Frontiers in Marine Science* Volume 11 - 2024 (2024). doi:10.3389/fmars.2024.1365366